

Efficient reachability set generation and storage using decision diagrams

Andrew S. Miner* Gianfranco Ciardo

Department of Computer Science
College of William and Mary
Williamsburg, VA, 23185
{asminer,ciardo}@cs.wm.edu

Abstract. We present a new technique for the generation and storage of the reachability set of a Petri net. Our approach is inspired by previous work on Binary and Multi-valued Decision Diagrams but exploits a concept of locality for the effect of a transition’s firing to vastly improve algorithmic performance. The result is a data structure and a set of manipulation routines that can be used to generate and store enormous sets extremely efficiently in terms of both memory and execution time.

Classification: Reachability set generation. System verification. Computer tools.

1 Introduction

The generation of the state space, or reachability set, $\mathcal{S}$ for a discrete-state model is an essential step in many types of studies. In the case of logical verification, the goal might be to ensure that no “bad” states satisfying certain boolean conditions can be reached from the initial state. In the case of Markov stochastic modeling, the reachable states determine the size and meaning of the probability vector computed as a result of a numerical solution. In either case, Petri nets (or stochastic Petri nets) are often the formalism of choice to describe such discrete-state models in a formal and compact way.

For the type of problems we consider, $\mathcal{S}$ is finite but its size is so large that its exploration and storage become formidable challenges. Implementors of computer tools for the analysis of stochastic Petri nets quickly found out that the effort to generate $\mathcal{S}$ and the infinitesimal generator matrix $\mathbf{Q}$ of the underlying model was often comparable to that of the numerical solution of the Markov chain. Analogously, $\mathcal{S}$ could easily require an amount of storage comparable to that of $\mathbf{Q}$. In practice, these tools could manage reachability sets ranging in size from 10^4 to perhaps 10^6 states, depending on the quality of the implementation and on the amount of primary memory available.

To move beyond these limitations, innovative approaches had to be employed. In Sect. 2, we briefly present two lines of work that are related to, and indeed

* A.S. Miner’s work was supported by fellowships from the NASA Graduate Student Research Program and the Virginia Space Grant Consortium

inspired, our research. One is represented by the 1994 paper by Pastor et al. [17], who proposed the use of Binary Decision Diagrams (BDDs) [2, 3] for the storage and generation of the reachability set for a safe (1-bounded) Petri net. By exploiting the compact representation of BDDs, they generated very large reachability sets in a matter of hours, with small storage requirements. In later work [15, 16], a more efficient encoding is introduced based on place invariants; however, the underlying logic is still based on binary variables. In our work, we instead use an extension of BDDs to non-binary logic, as proposed for the Multi-valued Decision Diagrams (MDDs) of Kam [11, 19], or the “shared trees” of Zampunièris [20]. Our title uses the term “decision diagrams” because the results we present, while particularly relevant to MDDs, apply also to BDDs.

We are also building upon our own previous work on state space storage [8]. By using a multilevel data structure based on a decomposition of a Petri net into submodels, we showed how $\mathcal{S}$ can be stored using little over a small integer (i.e., one or two bytes) per state. While this amount is linear in $|\mathcal{S}|$ (unlike the results achieved by Pastor with BDDs, which are sublinear for most models of interest), the reason for introducing our structure was that it fulfills many of the needs of numerical approaches based on Kronecker algebra [10, 12]: for this application, we have shown how it substantially improves the solution time for very large and sparse problems [5]. Furthermore, for exact numerical solution, one or two bytes per state represents a small fraction of the overall memory requirements, which include several single- or double-precision vectors of size $|\mathcal{S}|$. Another idea exploited in [8] was that of *event locality*: the realization that we can automatically detect *a priori* the identity of the submodels affected by a given event, so that only the corresponding “local substates” change when the event occurs. This concept of locality, appropriately extended when applied to MDDs, enables us to achieve great speedups.

Another related work is [4], which also considers a state space $\mathcal{S}$ defined as a subset of the cross-product $\mathcal{S}_K \times \cdots \times \mathcal{S}_1$ of arbitrary sets. Starting from the representation of $\mathcal{S}$ as a boolean vector of size $|\mathcal{S}_K| \cdots |\mathcal{S}_1|$ suggested in [13], common bit subvectors are merged. However, the intent in [4] is the Kronecker-based numerical solution of stochastic Petri nets, so no attempt is made to generate state spaces of the size considered in [17].

In the theoretical results of Sect. 3, we combine the main idea of BDDs (merging common subtrees in the data structure used to encode $\mathcal{S}$) with our idea of locality (limiting the computation and effect of a given event to the affected submarkings) and of model decomposition (defining a marking as a collection of submarkings, not necessarily from a safe net, encoded as a vector of small integer indices), to obtain a very efficient approach for the storage and generation of $\mathcal{S}$.

Sect. 4 explores some implementation issues that affect memory and time complexity. In Sect. 5, we apply our approach to various models previously considered in the literature. For the two applications reported in [17], our approach is much faster and more memory-efficient, hence it can generate much larger states spaces (9.18×10^{626} and 7.29×10^{62} states, respectively).

Finally, Sect. 6 states our conclusions and future research directions.

2 Background and related work

Our definition of Petri net is quite general, admitting inhibitor arcs, transition guards, and marking-dependent arc multiplicities [6]. Since any of these extensions achieves Turing-equivalence, we assume that the Petri net model has a finite reachability set $\mathcal{S}$. Formally, we represent a Petri net as a tuple $(\mathcal{P}, \mathcal{T}, I, O, H, g, \mathbf{m}^{[0]})$ where:

- $\mathcal{P} = \{p_1, \dots, p_{|\mathcal{P}|}\}$ is a finite set of places. A marking $\mathbf{m} \in \mathbb{N}^{|\mathcal{P}|}$ assigns a number of tokens to each place.
- $\mathcal{T} = \{t_1, \dots, t_{|\mathcal{T}|}\}$ is a finite set of transitions, with $\mathcal{P} \cap \mathcal{T} = \emptyset$.
- $I : \mathcal{P} \times \mathcal{T} \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \mathbb{N}$, $O : \mathcal{P} \times \mathcal{T} \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \mathbb{N}$, and $H : \mathcal{P} \times \mathcal{T} \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \mathbb{N} \cup \{\infty\}$ describe the marking-dependent multiplicities of the input, output, and inhibitor arcs.
- $g : \mathcal{T} \times \mathbb{N}^{|\mathcal{P}|} \rightarrow \{0, 1\}$ describes the transition guards.
- $\mathbf{m}^{[0]} \in \mathbb{N}^{|\mathcal{P}|}$ is the initial marking.

A transition t is enabled in $\mathbf{m}$, we write $Enabled(t, \mathbf{m})$, iff

$$g(t, \mathbf{m}) = 1 \wedge \forall p_i \in \mathcal{P}, I(p_i, t, \mathbf{m}) \leq \mathbf{m}_i \wedge H(p_i, t, \mathbf{m}) > \mathbf{m}_i.$$

Firing an enabled transition t in $\mathbf{m}$ leads to $\mathbf{n} = New(t, \mathbf{m})$, satisfying

$$\forall p_i \in \mathcal{P}, \mathbf{n}_i = \mathbf{m}_i - I(p_i, t, \mathbf{m}) + O(p_i, t, \mathbf{m}).$$

The reachability set $\mathcal{S}$ is then defined as the smallest subset of $\mathbb{N}^{|\mathcal{P}|}$ that contains $\mathbf{m}^{[0]}$ and is closed under the one-step reachability relation; that is, if $\mathbf{m} \in \mathcal{S}$, $Enabled(t, \mathbf{m})$, and $\mathbf{n} = New(t, \mathbf{m})$, then $\mathbf{n} \in \mathcal{S}$ as well.

Since our goal is the efficient generation and storage of $\mathcal{S}$, we briefly describe the traditional approach. An algorithm that iteratively builds $\mathcal{S}$, by processing one marking at a time, is shown in Fig. 1. Its execution time is then at least $O(|\mathcal{S}| \cdot |\mathcal{P}| \cdot |\mathcal{T}|)$. Indeed, it can be worse, since statement 8 requires to search for a newly generated state $\mathbf{n}$ in the set of currently known states. If $\mathcal{S}$ and $\mathcal{U}$ are stored using some type of search tree (e.g., AVL or Splay trees [7] or B-trees [1]), this implies an additional $\log |\mathcal{S}|$ factor.

Considering now the memory usage, a simple approach is to store $\mathcal{S}$ as a search tree where the keys are markings encoded as vectors of $|\mathcal{P}|$ natural numbers (or booleans, for safe nets). However, this requires $O(|\mathcal{P}| \cdot |\mathcal{S}|)$ bytes. Sparse techniques can be used to store the marking vectors, but these are only moderately beneficial, and only when most markings have many zero entries.

2.1 A multilevel data structure to store $\mathcal{S}$

Given a Petri net, we can partition its set of places $\mathcal{P}$ into K subsets $\mathcal{P}_k$, $k = K, \dots, 1$, and define $\mathcal{S}_k$ to be the set of reachable “local” submarkings for $\mathcal{P}_k$:

$$\mathcal{S}_k = \{\mathbf{m}_k : \exists \mathbf{m}_K, \dots, \mathbf{m}_{k+1}, \mathbf{m}_{k-1}, \dots, \mathbf{m}_1, [\mathbf{m}_K, \dots, \mathbf{m}_1] \in \mathcal{S}\}.$$

```

Explore( $\mathcal{P}, T, I, O, H, g, \mathbf{m}^{[0]}$ );
1.  $\mathcal{S} \leftarrow \emptyset$ ;
2.  $\mathcal{U} \leftarrow \{\mathbf{m}^{[0]}\}$ ;
3. while  $\mathcal{U} \neq \emptyset$  do
4.   choose a marking  $\mathbf{m}$  in  $\mathcal{U}$ ;
5.   move  $\mathbf{m}$  from  $\mathcal{U}$  to  $\mathcal{S}$ ;
6.   for each transition  $t$  such that  $Enabled(t, \mathbf{m})$  do
7.      $\mathbf{n} \leftarrow New(t, \mathbf{m})$ ;
8.     if  $\mathbf{n} \notin \mathcal{S} \cup \mathcal{U}$  then
9.        $\mathcal{U} \leftarrow \mathcal{U} \cup \{\mathbf{n}\}$ ;
10. return  $\mathcal{S}$ ;

```

• $\mathcal{S}$: markings explored so far
 • $\mathcal{U}$: markings found but not yet explored

Fig. 1. A procedure *Explore* to generate $\mathcal{S}$.

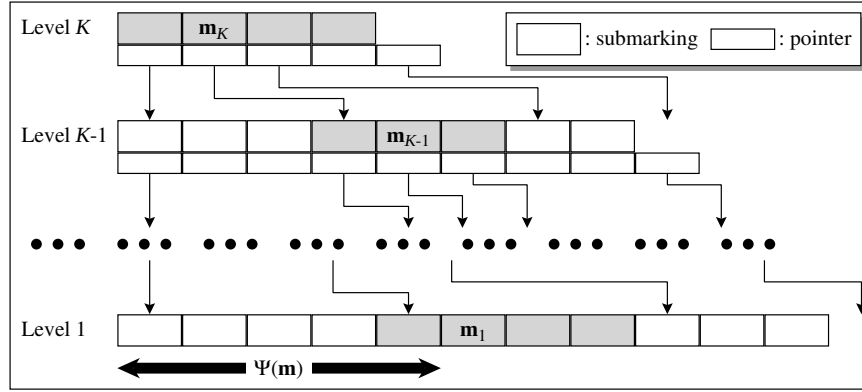


Fig. 2. An array implementation of the multilevel data structure in [8].

In [8], we discussed a data structure to store a subset $\mathcal{S}$ (the *actual state space*) of a cross-product $\mathcal{S}_K \times \dots \times \mathcal{S}_1$ of K sets (the *potential state space*). Fig. 2 illustrates the idea using arrays. At the top level, there is (at most) one instance of each submarking $\mathbf{m}_K \in \mathcal{S}_K$, and the corresponding pointer points to the submarkings for the remaining levels that can coexist with $\mathbf{m}_K$, that is, all $[\mathbf{m}_{K-1}, \dots, \mathbf{m}_1]$ such that $[\mathbf{m}_K, \dots, \mathbf{m}_1] \in \mathcal{S}$, and so on. To determine whether a given marking $\mathbf{m} = [\mathbf{m}_K, \dots, \mathbf{m}_1]$ is reachable, we search for $\mathbf{m}_K$ in the grayed-out portion of level K . If found, we follow the pointer identifying the grayed-out portion of level $K-1$, search that portion for $\mathbf{m}_{K-1}$, and repeat until either we find $\mathbf{m}_1$ in the grayed-out portion of level 1, or we fail to find a submarking $\mathbf{m}_k$, for some $k = K, \dots, 1$. In the former case, the marking $\mathbf{m}$ is reachable and the offset of the position where we found $\mathbf{m}_1$ in the array for level 1 indicates the lexicographic order $\Psi(\mathbf{m})$ of $\mathbf{m}$ in $\mathcal{S}$ (i.e., the number of reachable markings smaller than $\mathbf{m}$). In the latter case, $\mathbf{m}$ is not reachable.

In practice, it is much more efficient to use the index of a submarking in the data structure encoding $\mathcal{S}$, not the actual submarking, hence only $\lceil \log_2 |\mathcal{S}_k| \rceil$ bits are required for each submarking stored at level k . Assuming a sufficiently large branching factor from each level to the next, the memory requirements are

dominated by the bottom level, for which no pointers are required. Thus, we can store $\mathcal{S}$ in little over $|\mathcal{S}| \cdot \lceil \log_2 |\mathcal{S}_1| \rceil$ bits.

Arrays are used in [8] to reduce memory requirements *after* having completed the exploration of $\mathcal{S}$. However, *during* the exploration, the same approach requires a dynamic data structure allowing efficient search and insertion (e.g., an AVL or Splay tree) for each grayed-out portion at each level; we stress that, in this case, *a priori* knowledge of the sets $\mathcal{S}_k$ is not required, although a decision must be made about an upper bound on their size, so that indices can be stored. In practice, one, two, or four bytes are used (four bytes being rarely needed, as most local reachability sets contain no more than 2^8 or 2^{16} elements).

Another concept from [8] that inspired our current work is the *locality* of a transition. By examining the Petri net definition, it is possible to determine (conservatively) what places are affected when transition t fires. Then, we define the locality k^* of t as the largest k such that $\mathcal{P}_k$ contains an affected place, and we know that any submarking $\mathbf{m}_k$, for $k = K, \dots, k^* + 1$ is not affected by t .

By exploiting locality we achieve greater efficiency during both exploration of $\mathcal{S}$ and subsequent searches (needed, for example, in a Kronecker-based Markov analysis). If $\mathbf{n} = \text{New}(t, \mathbf{m})$, and we know the K pointers, or offsets into the K arrays, $\Psi_K(\mathbf{m}_K), \Psi_{K-1}(\mathbf{m}_K, \mathbf{m}_{K-1}), \dots, \Psi_1(\mathbf{m}_K, \dots, \mathbf{m}_1) = \Psi(\mathbf{m})$ for $\mathbf{m}$, the search for $\mathbf{n}$ into our multilevel data structure can start at level k^* , since we know that $\mathbf{n}_k = \mathbf{m}_k$, hence $\Psi_k(\mathbf{n}_K, \dots, \mathbf{n}_k) = \Psi_k(\mathbf{m}_K, \dots, \mathbf{m}_k)$, for $k = K, \dots, k^* + 1$.

2.2 Using BDDs to store $\mathcal{S}$

It is well-known that a boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ can be represented as a boolean vector of size 2^n indexed starting at zero, where the entry in position $b_n \dots b_1$, interpreted as an unsigned n -bit integer in base 2, is 0 iff $f(b_n, \dots, b_1) = 0$. However, this representation requires exponential space. BDDs were introduced [14] to alleviate this problem, as they allow one to encode and compute many boolean functions of interest in a very compact way.

A binary decision diagram (BDD) is a directed, acyclic graph with terminal nodes, labeled from the set $\{0, 1\}$, and non-terminal nodes, labeled from the set of variable names. Only non-terminal nodes have outgoing arcs, and each non-terminal node has exactly two outgoing arcs, labeled 0 and 1. Every non-terminal node in the graph represents some logic function f . A non-terminal node is represented by the tuple $(x, f_{x=0}, f_{x=1})$, where x is the variable name, the 0 and 1 arcs point to the cofactors $f_{x=0}$ and $f_{x=1}$, respectively (given a function f on variables $x_1, \dots, x_n$, the cofactors of f with respect to x_i are $f_{x_i=c} = f(x_1, \dots, x_{i-1}, c, x_{i+1}, \dots, x_n)$).

An ordered BDD (OBDD) has a total ordering on the variables such that any path of the graph must visit variables in that order. Finally, a reduced OBDD (ROBDD) has the following properties: there are at most two terminal nodes, with labels 0 and 1, there is no non-terminal node $(x, f_{x=0}, f_{x=1})$ with $f_{x=0}$ equal to $f_{x=1}$, and all non-terminals are unique, i.e., there are no two nodes $(x, f_{x=0}, f_{x=1})$ and $(x, g_{x=0}, g_{x=1})$ where $(f_{x=0} = g_{x=0}) \wedge (f_{x=1} = g_{x=1})$. Given a total ordering on the variables, ROBDDs are a canonical representation: two

$BDD_{\text{explore}}(\mathcal{P}, T, I, O, H, g, \mathbf{m}^{[0]});$	
1. $\mathcal{O} \leftarrow \emptyset;$ 2. $\mathcal{S} \leftarrow \{\mathbf{m}^{[0]}\};$ 3. while $\mathcal{S} \neq \mathcal{O}$ do 4. $\mathcal{O} \leftarrow \mathcal{S};$ 5. $\mathcal{S} \leftarrow \Delta(\mathcal{S}) \cup \mathcal{S};$ 6. return $\mathcal{S};$	• $\mathcal{O}$: old reachability set • $\mathcal{S}$: current reachability set

Fig. 3. A BDD-based procedure to generate $\mathcal{S}$.

logic functions f and g are equivalent iff f and g are represented by the same ROBDD. Bryant [2, 3] showed how ROBDDs can be efficiently manipulated. In the following we use “BDD” to mean “ROBDD”.

Pastor et al. [15–17] use BDDs for the generation and storage of the reachability set of a safe Petri net. In [17], the authors partition $\mathcal{P}$ into $K = |\mathcal{P}|$ subsets, with each subset containing a single place. In this case, each place can contain at most one token, so the simple encoding of a single BDD variable per place is used. A BDD is used to encode $\chi_{\mathcal{S}}$, the characteristic function of $\mathcal{S}$, defined by $\chi_{\mathcal{S}}(\mathbf{m}_{|\mathcal{P}|}, \dots, \mathbf{m}_1) = 1$ iff $\mathbf{m} \in \mathcal{S}$. Hence, we can talk equivalently of boolean functions or sets. The main result of [17] is that, given a set of markings $\mathcal{X}$, we can compute the BDD for the set $\Delta(\mathcal{X})$ of markings reachable from them in one firing, where the operator Δ is itself encoded as a BDD and can be built based on the Petri net definition. Fig. 3 illustrates the idea (we show a simplified version of the algorithm in [17]). Of course, the sets $\mathcal{O}$, $\mathcal{S}$, and $\Delta(\mathcal{S})$ are all encoded as BDDs. More sophisticated partitions and encodings are discussed in [15, 16], where invariants are used to reduce the number of boolean variables.

A fundamental property ensuring an efficient approach is that the number of iterations performed by BDD_{explore} is bounded by the *sequential depth* of the Petri net, that is, the maximum number of firings required to reach any marking starting from $\mathbf{m}^{[0]}$ (a quantity no larger than the diameter of the reachability graph). Thus, while each iteration usually implies a substantial computation, the number of iterations is usually quite small.

When the Petri net is not safe, the authors suggest two obvious ways to encode the number of tokens $\mathbf{m}_i$ of place p_i using booleans. If place p_i is k -bounded, we can use a *one-hot encoding* with k variables $b_i^1, \dots, b_i^k$, at most one of which will be nonzero in any marking (they are all zero iff p_i is empty), or a *binary encoding* with $\lceil \log_2(k+1) \rceil$ variables. The former results in more boolean variables overall, but also in a simpler encoding of the Δ function.

2.3 Multi-valued decision diagrams

BDDs have been generalized to integer functions on integer (instead of binary) variables, resulting in MDDs [11, 19] (see also the “shared trees” in [20]). MDDs can then represent functions of the form

$$\mathcal{S}_1 \times \mathcal{S}_2 \times \dots \times \mathcal{S}_n \rightarrow \{0, \dots, m-1\},$$

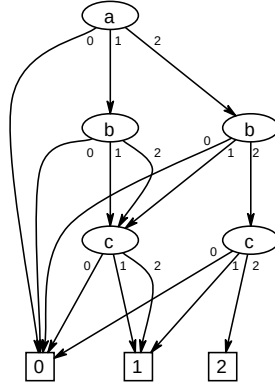


Fig. 4. MDD representing $\min(a, b, c)$

```

Case( $F, G^0, \dots, G^{m-1}$ )
1. if  $F$  is a constant  $r$  then return  $G^r$ ;
2. if  $G^0 = G^1 = \dots = G^{m-1}$  then return  $G^0$ ;
3. if  $G^0 = 0 \wedge \dots \wedge G^{m-1} = m-1$  then return  $F$ ;
4. if cache contains entry for  $(F, G^0, \dots, G^{m-1})$  then
5.   return cache entry result;
6. let  $x_k$  be the top variable of  $F, G^0, \dots, G^{m-1}$ ;
7. for  $i \leftarrow 0$  to  $N_k - 1$  do
8.    $H^i \leftarrow \text{Case}(F_{x_k=i}, G^0_{x_k=i}, \dots, G^{m-1}_{x_k=i})$ ;
9. if  $H^0 = H^1 = \dots = H^{N_k-1}$  then
10.   $R \leftarrow H^0$ ;
11. else
12.   $R \leftarrow \text{UniqueTableInsert}(x_k, H^0, \dots, H^{N_k-1})$ ;
13. add  $[(F, G^0, \dots, G^{m-1}), R]$  to cache;
14. return  $R$ ;

```

Fig. 5. Algorithm for the *Case* operator

where $\mathcal{S}_i = \{0, \dots, N_i - 1\}$. Non-terminal MDD nodes labeled with variable x_i have exactly N_i outgoing arcs, labeled 0 through $N_i - 1$; if f is the function represented by the node, we write it as $(x_i, f_{x_i=0}, \dots, f_{x_i=N_i-1})$. Terminal MDD nodes are labeled from the set $\{0, \dots, m-1\}$. The definitions for *ordered* and *reduced* MDDs are similar to those for BDDs. As with ROBDDs, given a total ordering on the variables, reduced ordered MDDs (ROMDDs) are a canonical representation. We use “MDD” to mean “ROMDD”.

An example is depicted in Fig. 4. The MDD shown represents the function $\min(a, b, c)$, where a, b, c can take on the values $\{0, 1, 2\}$. The MDD is reduced: no two nodes are equivalent and no node exists with all output arcs equal.

We can manipulate MDDs by using the *Case* operator, defined in [19] by $\text{Case}(F, G^0, \dots, G^{m-1}) = G^i$ if $F = i$, where the range of F is $\{0, \dots, m-1\}$. A recursive algorithm for computing the *Case* operator based on the relation

$$\text{Case}(F, G^0, \dots, G^{m-1})_{x=i} = \text{Case}(F_{x=i}, G^0_{x=i}, \dots, G^{m-1}_{x=i})$$

is given in Fig. 5. Given reduced MDDs, the algorithm returns a reduced MDD. The reductions are performed in line 10, which ensures that a node with equal arcs is not created, and in line 12, which ensures that two equivalent nodes are not created. Equivalent nodes are detected via a uniqueness table, usually implemented as a hash table. Before a new node is created, the uniqueness table is checked for an equivalent existing node. Another data structure used by an implementation of MDDs is a cache of operations. This prevents duplication of work, as a second call to *Case* with the same parameters will use the result saved in the cache. Both the node uniqueness table and the cache are well-known techniques that apply equally well to MDDs and BDDs [3, 19].

Like BDDs, MDDs can be used to represent a set $\mathcal{S}$ of integer tuples by storing the characteristic function $\chi_{\mathcal{S}}$ of the set. Sets can then be manipulated using MDD operations on their characteristic functions. For instance, the union of two sets is computed by $\chi_{\mathcal{A} \cup \mathcal{B}} = \text{Union}(\chi_{\mathcal{A}}, \chi_{\mathcal{B}}) = \text{Case}(\chi_{\mathcal{A}}, \chi_{\mathcal{B}}, 1)$ and the intersection of two sets is computed by $\chi_{\mathcal{A} \cap \mathcal{B}} = \text{Intersect}(\chi_{\mathcal{A}}, \chi_{\mathcal{B}}) = \text{Case}(\chi_{\mathcal{A}}, 0, \chi_{\mathcal{B}})$. In the remainder of the paper, we use the operators *Union* and *Intersect* for clarity, with the understanding that they are implemented using the *Case* operator. Also, we will sometimes write $\mathcal{S}$ instead of $\chi_{\mathcal{S}}$, with the understanding that $\mathcal{S}$ is always represented by its characteristic function $\chi_{\mathcal{S}}$.

3 Our technique

As in any structured approach, we assume that the partition of $\mathcal{P}$ into K sets $\mathcal{P}_K, \dots, \mathcal{P}_1$ has been performed according to some criterion. In our case, we require a “product-form” decomposition [9], that is:

1. There exist K functions $\text{Enabled}_k : \mathcal{T} \times \mathbb{N}^{|\mathcal{P}_k|} \rightarrow \{0, 1\}$ such that

$$\forall t \in \mathcal{T}, \text{Enabled}(t, \mathbf{m}) \Leftrightarrow \text{Enabled}_K(t, \mathbf{m}_K) \wedge \dots \wedge \text{Enabled}_1(t, \mathbf{m}_1).$$

2. There exist K functions $\text{New}_k : \mathcal{T} \times \mathbb{N}^{|\mathcal{P}_k|} \rightarrow \mathbb{N}^{|\mathcal{P}_k|}$ such that

$$\forall t \in \mathcal{T}, \mathbf{n} = \text{New}(t, \mathbf{m}) \Leftrightarrow \mathbf{n}_K = \text{New}_K(t, \mathbf{m}_K) \wedge \dots \wedge \mathbf{n}_1 = \text{New}_1(t, \mathbf{m}_1).$$

Such a partition can be found automatically, from a simple inspection of the marking-dependent expressions for I , O , H , and g . Assuming that each $g(t, \cdot)$ is expressed as a conjunction of terms, $g(t, \cdot) = f_1(t, \cdot) \wedge \dots \wedge f_{r_t}(t, \cdot)$, and letting $\mathcal{G}$ be the union of these terms, over all transitions, algorithm *Partition* can be used for this purpose (Fig. 6). In particular, if I , O , and H are not marking-dependent and g is identically equal 1, the previous two criteria are satisfied by the finest partition, where $K = |\mathcal{P}|$, i.e., each place is in a class by itself. Of course, any coarsening of a product-form partition is itself a product-form partition, so it could be used as well. We illustrate the effect of this choice in Sect. 5, but finding a “good” partition is still an open problem.

A concept of locality for both the enabling condition and the effect of firing, more refined than the one we introduced in [8], is expressed by making use of the functions just introduced. A transition t is local to exactly $\{\mathcal{P}_{k_1}, \dots, \mathcal{P}_{k_n}\}$ if

$$\forall k \notin \{k_1, \dots, k_n\}, \forall \mathbf{m}_k \in \mathcal{S}_k, \text{Enabled}_k(t, \mathbf{m}_k) = 1 \wedge \text{New}_k(t, \mathbf{m}_k) = \mathbf{m}_k$$

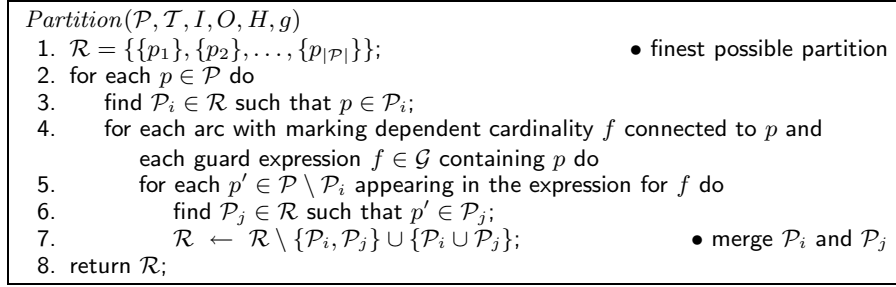


Fig. 6. Algorithm to find the finest product-form partition $\mathcal{R} = \{\mathcal{P}_1, \dots, \mathcal{P}_K\}$ of $\mathcal{P}$.

(i.e., if $k \notin \{k_1, \dots, k_n\}$, the marking of $\mathcal{S}_k$ does not affect the enabling, nor is it affected by the firing, of t).

We are now ready to discuss state space generation using MDDs. Since $\mathcal{P}$ has been partitioned into K sets, we use a K -variable MDD to store $\mathcal{S}$. As discussed in Sect. 2, a submarking $\mathbf{m}_k \in \mathcal{S}_k \subset \mathcal{I}^{|\mathcal{P}_k|}$ can be indexed by a (small) integer $m_k \in \{0, 1, \dots, |\mathcal{S}_k| - 1\}$. In the following, and in our implementation, we use this encoding for both simplicity and efficiency. Henceforth, we write $Enabled_k(t, m_k)$ and $n_k = New_k(t, m_k)$ instead of $Enabled_k(t, \mathbf{m}_k)$ and $\mathbf{n}_k = New_k(t, \mathbf{m}_k)$, to stress that we are operating on (local) indices, but we will keep talking about (sub)markings, with the understanding that only indices are really stored in our MDDs.

To generate the reachability set $\mathcal{S}$, we must manipulate the MDD representation of $\mathcal{S}$ to simulate the firing of transitions. We first show how to do this for *local* transitions (local to exactly one $\mathcal{P}_k$), and then for *synchronizing* transitions (local to more than one set of places).

3.1 Local transition firing

A transition t local to exactly one set of places $\mathcal{P}_k$ has the special property that t only affects places in $\mathcal{P}_k$. More formally, this says that $Enabled(t, \mathbf{m}) = Enabled_k(t, m_k)$ and $New(t, \mathbf{m}) = [m_K, \dots, m_{k+1}, New_k(t, m_k), m_{k-1}, \dots, m_1]$. This implies that, for any reachable marking $[\alpha, m_k, \beta]$ and any local transition t such that $Enabled_k(t, m_k)$, marking $[\alpha, New_k(t, m_k), \beta]$ is also reachable. Thus, the markings $[\alpha, New_k(t, m_k), \beta]$ must be added to $\mathcal{S}$, through the arc update

$$f_{x_k = New_k(t, m_k)} \leftarrow Union(f_{x_k = m_k}, f_{x_k = New_k(t, m_k)}), \quad (1)$$

for each MDD node f in $\chi_{\mathcal{S}}$ labeled with variable x_k . After performing this operation, for any reachable marking $[\alpha, m_k, \beta]$, the marking $[\alpha, New_k(t, m_k), \beta]$ is now reachable. To see this, consider the path in $\chi_{\mathcal{S}}$ for $[\alpha, m_k, \beta]$ ending at terminal node 1. If the path contains a node labeled with x_k , then the update ensures that there is also a path for $[\alpha, New_k(t, m_k), \beta]$ ending at 1, found by following the downward arc $New_k(t, m_k)$ instead of m_k at node x_k . Otherwise,

```

DoLocals( $\mathcal{S}$ )
1. if  $\mathcal{S} = 0$  return 0;
2. if  $\mathcal{S} = 1$  return 1;
3. if cache contains the entry for  $\mathcal{S}$  then
4.   return cache entry result;
5. let  $x_k$  be the top variable of  $\mathcal{S}$ ;
6.  $Changed \leftarrow \emptyset$ ;
7. for  $i \leftarrow 0$  to  $N_k - 1$  do
8.    $\mathcal{H}^i \leftarrow DoLocals(\mathcal{S}_{x_k=i})$ ;
9.   if  $\mathcal{H}^i \neq 0$  then  $Changed \leftarrow Changed \cup \{i\}$ ;
10. while  $Changed \neq \emptyset$  do
11.   remove some element  $i$  from  $Changed$ ;
12.   for each transition  $t$  local to  $\mathcal{P}_k$  do
13.     if  $Enabled_k(t, i)$  then
14.        $j \leftarrow New_k(t, i)$ ;
15.        $\mathcal{F} \leftarrow Union(\mathcal{H}^i, \mathcal{H}^j)$ ;
16.       if  $\mathcal{F} \neq \mathcal{H}^j$  then
17.          $Changed \leftarrow Changed \cup \{j\}$ ;
18.          $\mathcal{H}^j \leftarrow \mathcal{F}$ ;
19. if  $\mathcal{H}^0 = \mathcal{H}^1 = \dots = \mathcal{H}^{p-1}$  then
20.    $\mathcal{R} \leftarrow \mathcal{H}^0$ ;
21. else
22.    $\mathcal{R} \leftarrow UniqueTableInsert(x_k, \mathcal{H}^0, \dots, \mathcal{H}^{p-1})$ ;
23. add  $[\mathcal{S}, \mathcal{R}]$  to cache;
24. return  $\mathcal{R}$ ;

```

• Application of Eq. 1

Fig. 7. Algorithm for firing local transitions

the path does not contain a node labeled with x_k , and thus does not depend on variable x_k ; in this case, the path for $[\alpha, x_k, \beta]$ ends at 1 for any value of x_k .

The operation performed by Eq. 1 adds to $\mathcal{S}$ all the markings reached by firing a single local transition t when the submarking for $\mathcal{S}_k$ is m_k . To completely simulate the firing of transition t , we perform this operation for all submarkings m_k such that $Enabled_k(t, m_k)$. This can be done “in parallel”: each MDD node in $\chi_{\mathcal{S}}$ labeled with variable x_k is visited once, and Eq. 1 is applied to multiple submarkings m_k . In fact, we can perform Eq. 1 for all transitions local to $\mathcal{P}_k$ in one operation.

This idea is the basis for our local transition manipulation algorithm, shown in Fig. 7. Given a set of markings $\mathcal{S}$, *DoLocals* returns the set of markings that can be reached by a marking in $\mathcal{S}$ firing any sequence of local transitions (including none). The algorithm visits each MDD node in $\chi_{\mathcal{S}}$ and, based on the variable label x_k of the node, fires transitions local to $\mathcal{P}_k$ using Eq. 1. Arcs that change are added to the set *Changed*, and are explored again.

3.2 Synchronizing transition firing

A transition t local to more than one $\mathcal{S}_k$ requires to check more than one set of places to determine its enabling and the markings reached when it fires. If t is local to $\mathcal{P}_{k_1}, \dots, \mathcal{P}_{k_n}$, then $Enabled(t, \mathbf{m}) = Enabled_{k_1}(t, m_{k_1}) \wedge \dots \wedge$

$Enabled_{k_n}(t, m_{k_n})$. We cannot simulate the firing of a synchronizing transition t by examining MDD nodes in isolation as we did with local transitions. Instead, to add the markings reached when t fires, we must perform three distinct operations: determine the set of markings that enable t , determine the new markings reached after t fires, and add the new markings to the set of reachable markings.

Given t , we compute $\mathcal{E}(t)$, the set of potential markings enabling t , as

$$\chi_{\mathcal{E}(t)}(x_K, \dots, x_1) = Enabled_K(t, x_K) \wedge \dots \wedge Enabled_1(t, x_1).$$

Assuming the sets $\mathcal{S}_k$ can be generated *a priori*, the sets $\mathcal{E}(t)$ can also be computed once *a priori* and used throughout the entire state space generation process. Otherwise, $\mathcal{E}(t)$ must be recomputed whenever a new marking is added to $\mathcal{S}_k$. Either way, once we have obtained $\mathcal{E}(t)$, we can compute the set of markings in $\mathcal{S}$ that enable t as $Intersect(\mathcal{S}, \mathcal{E}(t))$.

Then, we simulate the firing of t by updating the submarkings affected by t . For each marking $[m_K, \dots, m_1]$ that enables t , the marking $[m'_K, \dots, m'_1]$ is reached after t fires, where $m'_k = New_k(t, m_k)$ if t is local to $\mathcal{P}_k$, $m'_k = m_k$ otherwise. This set of markings is computed using the *Fire* operator, whose algorithm is shown in Fig. 8. Given a set of markings that enable t , *Fire* returns the set of markings reached after t fires by copying the downward arc of m_k from the input set to $New_k(t, m_k)$ in the output set (line 12). Recall that $New_k(t, m_k) = m_k$ for all m_k if t is not local to $\mathcal{P}_k$; thus, *Fire* performs a simple copy in this case. The *Union* operator in line 12 is required because, with marking-dependent arc cardinalities, the firing of t in multiple submarkings i might lead to the same submarking j . Note that the *Fire* operator requires us to specify the parameter x_k , which represents the “current variable”, because it may change “don’t care” variables. This can occur if every submarking enables t but not every submarking is reached by firing t . Finally, *Fire* returns $\mathcal{S}$ unchanged if the top variable x_k is past $Last(t) = \min\{k : t \text{ is local to } \mathcal{P}_k\}$, the last submarking affected by t .

3.3 Our generation algorithm

The generation algorithm is shown in Fig. 9. It consists of two phases per iteration. The first phase, represented by line 3, finds all submarkings that can be reached from the current $\mathcal{S}$ by firing only sequences of local transitions. The second phase, represented by lines 5 through 9, handles the synchronizing transitions.

Thus, the number of iterations required by *MDDexplore* is bounded by one (to recognize that $\mathcal{S}$ has not changed) plus the *synchronizing depth* of the net (as opposed to the sequential depth of [17]) defined as $\max_{\mathbf{m} \in \mathcal{S}} \{d(\mathbf{m})\}$, where $d(\mathbf{m}) = \min\{n \in \mathbb{N} : \exists (t_1^1, \dots, t_{l_1}^1, y^1, t_1^2, \dots, t_{l_2}^2, y^2, \dots, y^n, t_1^{n+1}, \dots, t_{l_{n+1}}^{n+1}) \in \mathcal{T}^*,$
with $l_i \geq 0$, t_j^i local transitions and y^i synchronizing transitions,
whose firing leads from $\mathbf{m}^{[0]}$ to $\mathbf{m}\}$.

The actual number of iterations can be smaller, depending on the order in which synchronizing transitions are processed: transition y^2 in the sequence

```

Fire( $x_k, \mathcal{S}, t$ )
1. if  $\mathcal{S} = 0$  return 0;
2. if  $k < \text{Last}(t)$  return  $\mathcal{S}$ ;           • if  $k < \text{Last}(t)$  then  $t$  does not affect  $\mathcal{S}$ 
3. if cache contains entry for  $(x_k, \mathcal{S}, t)$  then
4.   return cache entry result;
5. let  $x_{k'}$  be the top variable of  $\mathcal{S}$ ;
6. while  $(t$  not local to  $\mathcal{P}_k)$  and  $(k > k')$  do
7.    $k \leftarrow k - 1$ ;
8. for  $i \leftarrow 0$  to  $N_k - 1$  do
9.    $\mathcal{H}^i \leftarrow 0$ 
10. for  $i \leftarrow 0$  to  $N_k - 1$  do
11.    $j \leftarrow \text{New}_k(t, i)$ ;
12.    $\mathcal{H}^j \leftarrow \text{Union}(\mathcal{H}^j, \text{Fire}(x_{k-1}, \mathcal{S}_{x_k=i}, t))$ ;
13. if  $\mathcal{H}^0 = \mathcal{H}^1 = \dots = \mathcal{H}^{N_k-1}$  then
14.    $\mathcal{R} \leftarrow \mathcal{H}^0$ ;
15. else
16.    $\mathcal{R} \leftarrow \text{UniqueTableInsert}(x_k, \mathcal{H}^0, \dots, \mathcal{H}^{N_k-1})$ ;
17. add  $[(x_k, \mathcal{S}, t), \mathcal{R}]$  to cache;
18. return  $\mathcal{R}$ ;

```

Fig. 8. Algorithm for firing synchronizing transitions

```

MDDexplore( $\mathbf{m}^{[0]}$ )
1.  $\mathcal{S} \leftarrow (x_K = m_K^{[0]}) \wedge \dots \wedge (x_1 = m_1^{[0]})$ ;           • Set  $\mathcal{S}$  to the initial marking
2. repeat forever
3.    $\mathcal{S} \leftarrow \text{DoLocals}(\mathcal{S})$ ;
4.    $\mathcal{O} \leftarrow \mathcal{S}$ ;           • Save old set of reachable states
5.   for each synchronizing transition  $t$  do
6.      $\mathcal{E} \leftarrow \text{Intersect}(\mathcal{S}, \mathcal{E}(t))$ ;           •  $\mathcal{E}$  is the set of markings that enable  $t$ 
7.      $\mathcal{F} \leftarrow \text{Fire}(x_K, \mathcal{E}, t)$ ;           •  $\mathcal{F}$  is the set of markings reached after  $t$  fires
8.      $\mathcal{S} \leftarrow \text{Union}(\mathcal{S}, \mathcal{F})$ ;           • Add  $\mathcal{F}$  to  $\mathcal{S}$ 
9.   if  $\mathcal{O} = \mathcal{S}$  return  $\mathcal{S}$ ;

```

Fig. 9. Algorithm to compute $\mathcal{S}$

(y^1, y^2) leading from $\mathbf{m}^{[0]}$ to $\mathbf{m}$ is considered during the first iteration if y^1 is processed before y^2 , during the second iteration otherwise ([18] already recognized the importance of a good “chaining” order).

3.4 Logical queries on the state space

Once $\mathcal{S}$ has been generated, it can be used to answer various classes of logical queries, by performing MDD operations. For instance, suppose we want to compute the set of reachable markings satisfying some boolean condition q . To do this, we first build the set $\mathcal{Q}$ of potential markings in $\mathcal{S}_K \times \dots \times \mathcal{S}_1$ that satisfy q , and then compute $\mathcal{S} \cap \mathcal{Q}$. First, let us consider a simple query whose condition is enforced at a single level k . Given a condition $f_k : \mathcal{S}_k \rightarrow \{0, 1\}$, we compute the set of submarkings $\mathcal{Q}$ that satisfy f_k :

$$\chi_{\mathcal{Q}}(x_K, \dots, x_1) = f_k(x_k).$$

Complex queries are answered by combining simple queries. An important example is asking for the set of reachable (absorbing, or dead) markings that do not enable any transition. First, we compute $\mathcal{Q}(t)$, the set of potential markings that do not enable t by

$$\chi_{\mathcal{Q}(t)}(x_K, \dots, x_1) = (\text{Enabled}_K(t, x_K) = 0) \vee \dots \vee (\text{Enabled}_1(t, x_1) = 0)$$

for all transitions t . Next, we compute the set of potential markings that do not enable any transition, $\mathcal{Q} = \bigcap_{t \in \mathcal{T}} \mathcal{Q}(t)$. Finally, $\mathcal{S} \cap \mathcal{Q}$ gives us the reachable absorbing markings.

Another important class of queries deals with the possibility of a condition b occurring after a condition a has occurred. To answer this question we build the subsets $\mathcal{Q}_a$ and $\mathcal{Q}_b$ of $\mathcal{S}_K \times \dots \times \mathcal{S}_1$ satisfying a or b , respectively. Then, we run *MDDexplore* except that, in line 1, $\mathcal{S}$ is initialized to the set $\mathcal{S} \cap \mathcal{Q}_a$ of reachable markings satisfying a , instead of the initial marking. The intersection of the resulting $\mathcal{S}$ with $\mathcal{Q}_b$ gives exactly the set of markings satisfying b that can be reached from some reachable marking satisfying a . In the worst case, this approach requires as many iterations as for the original generation of $\mathcal{S}$.

4 Implementation issues

One way to implement a MDD data structure is to map each MDD node onto a BDD structure using some encoding. With this approach, MDD operators are translated into equivalent operations on the underlying BDD. Indeed, both Kam’s PhD thesis [11] and the timing results in [19] seem to suggest that this achieves high(er) efficiency. Instead, we choose to implement MDDs directly, where each MDD node contains some node information (the variable index, the number of downward pointers, etc.) and an array of downward pointers. As the next section shows experimentally, our approach can be vastly superior (at least for the examples we used). In the the first two applications, assigning one (safe) place per level effectively reduces our implementation to the BDD case, and doing so results in much higher execution times (even if locality is exploited in either case) and memory consumption.

To conserve memory in our implementation, MDD nodes store node indices instead of full pointers. In particular, MDD nodes labeled with variable x_1 can only have downward pointers to terminal nodes 0 and 1; thus, an array of bits is used. More sophisticated MDD node storage schemes would be possible, including sparse array storage and other forms of compression. This is because we do not directly modify MDD nodes: we use temporary full integer arrays during MDD node construction (e.g., for storing $H^0, \dots, H^{N_k-1}$ in algorithm *Case*), which are copied into appropriately-sized structures for long-term storage. In our experience, this compression introduces a 10%-20% CPU overhead, but it typically reduces memory usage by a factor of 4.

In our studies, we have assumed that the local reachability sets $\mathcal{S}_k$ (or supersets of them) are computable in isolation, i.e., before the generation of the overall reachability set $\mathcal{S}$. This is a common assumption with structured approaches

(e.g., [4, 13]), and certainly with safe nets, for which $\mathcal{S}_k \subseteq \{0, 1\}^{|\mathcal{P}_k|}$ by definition. This restriction can be lifted by dynamically generating local reachability sets $\mathcal{S}_k$ during generation of $\mathcal{S}$. This requires a more complex implementation of $Enabled_k$ and New_k , where the first time $New_k(t, \mathbf{m}_k)$ is called on $\mathbf{m}_k$, we compute the reached submarking $\mathbf{n}_k$ and add it to $\mathcal{S}_k$, if necessary.

We conclude this section with an observation. At each iteration, algorithm *MDDexplore* considers the firing of sequences of local transitions before examining the synchronizing transition. This is in contrast to the algorithm proposed in [17], which does not exploit the concept of locality, and is one of the reasons for our greater efficiency. As exploring the firing for local transitions is considerably less expensive than for the synchronizing transitions, we achieve two goals: we reduce the number of iterations required by *MDDexplore* (compare the sequential depth of [17] with our synchronizing depth) and we add more markings to $\mathcal{S}$ every time we explore the firing of a synchronizing transition (since we fire it from a set $\mathcal{S}$ already augmented by the local firings).

5 Results

We now apply our approach to various models taken from the literature. We examine first the dining philosophers and the slotted ring models presented in [17]. These are safe Petri nets composed of several identical subnets: the state space size is increased by adding subnets. Then, we consider the flexible manufacturing and kanban systems presented in [8]. These models have a fixed number of places and transitions: the state space size increases with the initial marking.

We implemented our approach in the tool SMART [7]. Our results are obtained on a 400 MHz Pentium II workstation under the Linux operating system. No run made use of virtual memory. For each experiment, Table 1 reports:

- The size of the state space $\mathcal{S}$.
- The final and peak number of MDD nodes (our data structure grows and contracts during the execution of *MDDexplore*).
- The final and peak memory consumption, in bytes (peak memory is an indicator of the amount of RAM required to avoid relying on virtual memory, while final memory is of interest in case $\mathcal{S}$ is saved for further use).
- The number of iterations performed by *MDDexplore*.
- The overall CPU times, in seconds.

We also explore the effect of different partitioning of the model into levels, in Fig. 11. Occasional “bumps” in the plots are artifacts of the compression used in our implementation.

5.1 Dining Philosophers

The dining philosopher model is composed of N subnets. The Petri net for the i^{th} philosopher is shown in Fig. 10(a). The net represents a philosopher and the philosopher’s right fork. The philosopher’s left fork, represented by the dotted

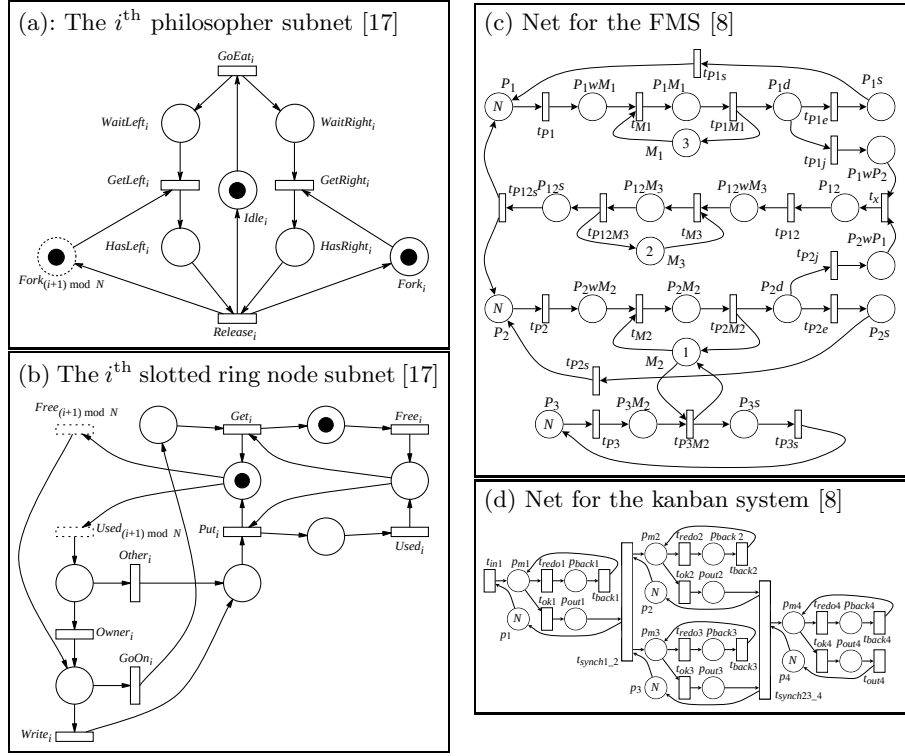


Fig. 10. The four models used in our experiments.

place $Fork_{(i+1) \bmod N}$, is part of the subnet for the next philosopher; it is depicted to illustrate how the subnets interact.

A natural partitioning scheme is to assign each philosopher to a separate level. We also investigate grouping two or three adjacent philosophers together in the same level. Table 1 shows the results for two philosophers per level. We find that grouping more than two philosophers in each level results in worse memory usage and CPU times (Fig. 11). However, there is an interesting tradeoff between having one or two philosophers per level: the former choice results in higher execution times, but lower memory requirements. The grouping where each place is in a different level corresponds to the BDD approach in [17], and it is vastly less efficient.

“True” MDDs with locality are extremely efficient for this example, both in terms of memory usage and generation time. There are two main reasons for this. First, our approach requires only two iterations, no matter how many philosopher subnets are present. The synchronizing depth of the model grows as N , since the synchronizing transitions are $GetLeft_i$, $GetRight_i$, and $Release_i$, and the farthest markings are those where N forks are taken, as left or right forks. However, any such marking $\mathbf{m}$ can be reached from $\mathbf{m}^{[0]}$ through $N!$

different firing sequences (if we ignore the position of local transition $GoEat_i$ in the sequence), and there is always one sequence that respects the order in which the synchronizing transitions are considered in *MDDexplore*. In other words, the entire reachability set has been discovered by the second execution of statement 3 in *MDDexplore*. In contrast, the number of iterations reported for the BDD approach grows as $2N + 1$ [17].

Second, the MDD contains exactly four distinct nodes at each level (except for the top level, which always has one), no matter how many philosophers we add or how many (adjacent) philosophers we group into each level. This is because a philosopher will either hold both forks, only the left fork, only the right fork, or no fork at all. This is still the case for a level containing several adjacent philosophers, where the left and right forks are the boundary forks between levels. Hence, the final number of MDD nodes in Table 1 is $4(N/2 - 1) + 1 = 2N - 3$. It is interesting to note that the peak number of nodes is also linear, $6N - 15$.

5.2 Slotted ring network

The Petri net for a single node of a slotted ring network protocol is shown in Fig. 10(b). The overall model is composed of N such subnets connected by sharing transitions ($Free_{(i+1) \bmod N}$ and $Used_{(i+1) \bmod N}$). Table 1 shows the results for a decomposition where each node is in a different level. The effect of other choices, one place per level (essentially a BDD) and two nodes per level, is as for the previous model (Fig. 11). The number of iterations required by *MDDexplore* is $N/2 + 2$, while that for the BDD approach in [17] grows quadratically.

5.3 Flexible Manufacturing System

The FMS model shown in Fig. 10(c) [8] is parameterized by the initial number N of tokens in P_1 , P_2 , and P_3 . We compare three different partitioning schemes in Fig. 11. The model is partitioned into 4, 6, or 19 levels. In the first case, the partition is $\{P_1, P_1wM_1, P_1M_1, M_1, P_1d, P_1s, P_1wP_2\}$, $\{P_{12}, P_{12}wM_3, P_{12}M_3, M_3, P_{12}s\}$, $\{P_2, P_2wM_2, P_2M_2, M_2, P_2d, P_2s, P_2wP_1\}$, and $\{P_3, P_2M_2, P_3s\}$. In the second case, it is $\{P_{12}, P_{12}wM_3, P_{12}M_3, M_3, P_{12}s\}$, $\{P_1, P_1wM_1, P_1M_1, M_1\}$, $\{P_1d, P_1s, P_1wP_2\}$, $\{P_2, P_2wM_2, P_2M_2, M_2\}$, $\{P_2d, P_2s, P_2wP_1\}$, and $\{P_3, P_2M_2, P_3s\}$. Finally, the partition with 19 levels is obtained by assigning each place to a different level, with the exception of the complementary places M_1 , M_2 , and M_3 , placed in the same level as the places P_1M_1 , P_2M_2 , and $P_{12}M_3$, respectively.

In this model, the effect of the partition choice is extremely noticeable in terms of both memory and execution times, and the finest partition is by far the best. Table 1 reports the detailed results for the 19-level partition. In this case, *MDDexplore* requires $N + 5$ iterations ($N + 1$ and $N + 2$ are instead required with the 4-level and 6-level partitions, respectively).

5.4 Kanban system

Fig. 10(d) shows the Petri net of a Kanban system [8]. This model is parameterized by the number of tokens N initially in p_1 , p_2 , p_3 , and p_4 . Also for this

	N	$ \mathcal{S} $	MDD Nodes		Memory (bytes)		# of Iters.	CPU (secs)
			Final	Peak	Final	Peak		
Philosophers (2 phils/level)	10	1.86×10^6	17	45	1,046	2,934	2	0.14
	20	3.46×10^{12}	37	105	2,446	7,134	2	0.37
	50	2.23×10^{31}	97	285	6,646	21,498	2	1.61
	100	4.97×10^{62}	197	585	15,788	58,164	2	5.67
	200	2.47×10^{125}	397	1,185	41,044	128,556	2	21.73
	300	1.23×10^{188}	597	1,785	65,124	197,604	2	47.12
	400	6.10×10^{250}	797	2,385	87,608	265,308	2	89.66
	500	3.03×10^{313}	997	2,985	110,008	332,508	2	134.43
	600	1.51×10^{376}	1,197	3,585	133,080	400,380	2	205.44
	700	7.48×10^{438}	1,397	4,185	155,648	467,748	2	296.31
	800	3.72×10^{501}	1,597	4,785	177,796	534,612	2	404.06
Slotted Ring (1 node/level)	900	1.85×10^{564}	1,797	5,385	200,280	601,980	2	498.25
	1000	9.18×10^{626}	1,997	5,985	222,764	669,180	2	630.40
	10	8.29×10^9	60	691	3,186	38,527	7	1.80
	20	2.73×10^{20}	220	4,546	12,601	263,002	12	41.13
	30	1.04×10^{31}	480	15,101	27,741	875,327	17	297.97
	40	4.16×10^{41}	840	37,066	48,606	2,149,372	22	1309.66
	50	1.72×10^{52}	1,300	76,308	85,486	5,342,493	27	4282.80
	60	7.29×10^{62}	1,860	139,852	149,106	11,902,318	32	11948.90
	5	2.90×10^6	149	433	7,214	21,190	10	0.62
	10	2.50×10^9	354	1,038	22,210	65,826	15	2.65
	15	2.17×10^{11}	634	1,868	48,600	144,656	20	7.03
FMS (19 levels)	20	6.03×10^{12}	989	2,923	89,411	266,707	25	14.77
	25	8.54×10^{13}	1,419	4,203	147,632	440,968	30	29.75
	50	4.24×10^{17}	4,694	13,978	804,785	2,410,321	55	260.02
	75	6.98×10^{19}	9,844	29,378	2,347,088	7,034,824	80	1047.90
	100	2.70×10^{21}	16,869	50,403	5,149,541	15,439,477	105	2845.66
	5	2.55×10^6	7	47	294	3,458	11	0.15
	10	1.01×10^9	12	87	1,018	22,318	21	1.83
Kanban (4 levels)	15	4.70×10^{10}	17	127	2,924	85,768	31	9.72
	20	8.05×10^{11}	22	167	7,049	238,473	41	33.96
	25	7.68×10^{12}	27	207	14,692	540,628	51	92.69
	50	1.04×10^{16}	52	407	197,687	9,510,714	101	2383.10
	75	7.83×10^{17}	77	607	877,068	51,881,884	151	17715.40

Table 1. Results for our models.

model we compare different partitioning schemes, in Fig. 11, corresponding to either 4 or 16 levels. The former case is as indicated by the subscripts 1, 2, 3, and 4 in Fig. 10(d), while the latter case assigns one place to each level. In this case, unlike the FMS, the finer partition is much worse than the coarser one. Table 1 shows the results for the 4-level partition. In this case, *MDDexplore* requires $2N + 1$ iterations, while the 16-level partition requires 5 iterations for $N = 1$ and $3N + 1$ iterations for $N > 1$.

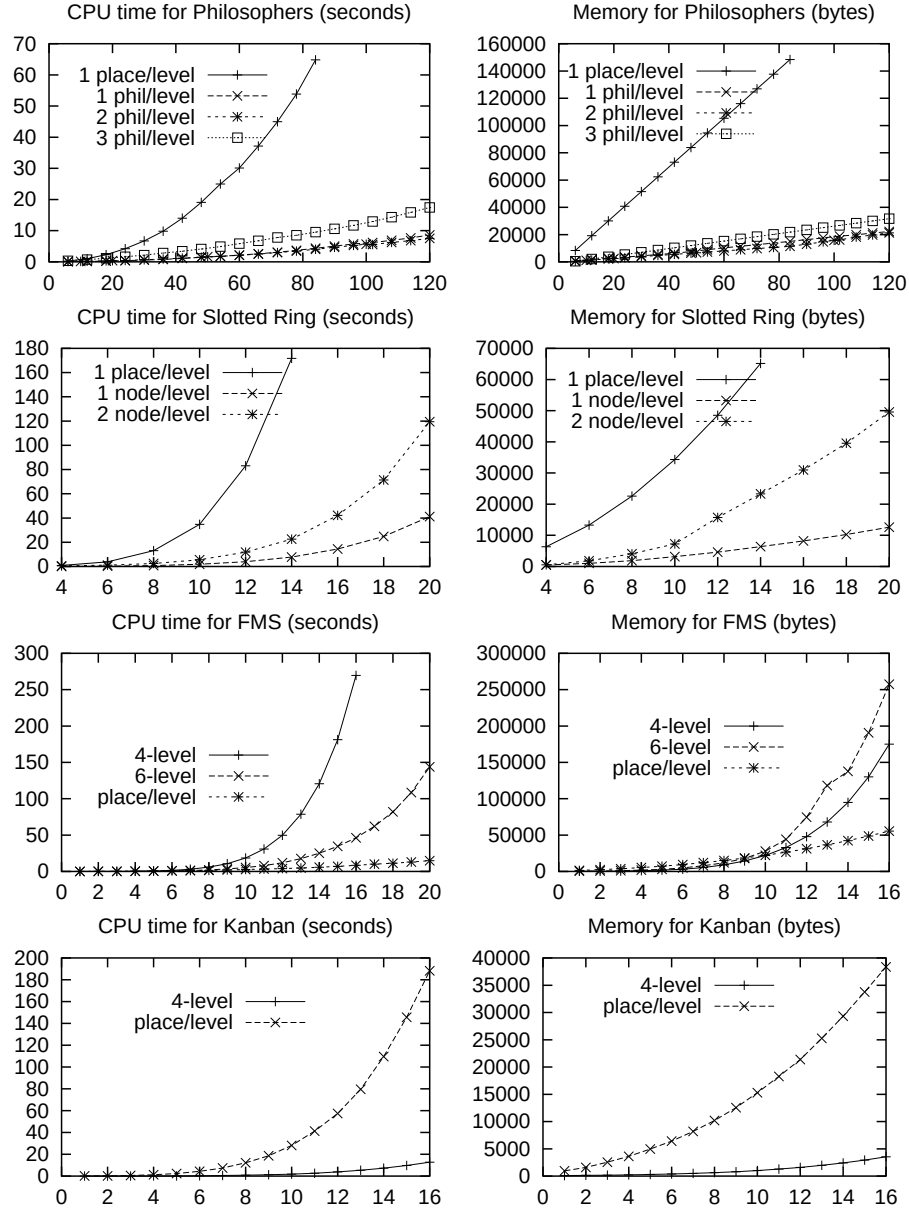


Fig. 11. Effect of different partitions on our models (N on the horizontal axis).

6 Conclusion

We presented a new technique for the generation and storage of the reachability set of a Petri net, closely related to recently proposed BDD-based approaches.

However, our use of multi-valued (not boolean) sets and the exploitation of locality to reduce both the number of iterations and the cost of each iteration in the generation procedure result in the ability to tackle much larger reachability sets than previously possible.

The application of our results goes beyond that of Petri net analysis, as it widens the size of the discrete-state systems for which an exhaustive logical verification might be reasonably attempted.

Much work remains to be done, however. We have demonstrated how the choice for the partition of the Petri net places (i.e., the decomposition of the discrete-state model), the order in which the levels are considered, and the order in which the synchronizing transitions are processed can have a substantial effect on the memory and time requirements of the approach. The existence of an efficient algorithm that can derive an optimal strategy for these choices is unlikely, but we shall seek heuristics that work well on models of practical interest.

References

1. S. C. Allmaier, M. Kowarschik, and G. Horton. State space construction and steady-state solution of GSPNs on a shared-memory multiprocessor. In *Proc. 7th Int. Workshop on Petri Nets and Performance Models (PNPM'97)*, pages 112–121, St. Malo, France, June 1997. IEEE Comp. Soc. Press.
2. R. E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Trans. Comp.*, 35(8):677–691, Aug. 1986.
3. R. E. Bryant. Symbolic boolean manipulation with ordered binary-decision diagrams. *ACM Comp. Surv.*, 24(3):393–318, 1992.
4. P. Buchholz. Hierarchical structuring of superposed GSPNs. In *Proc. 7th Int. Workshop on Petri Nets and Performance Models (PNPM'97)*, pages 81–90, St. Malo, France, June 1997. IEEE Comp. Soc. Press.
5. P. Buchholz, G. Ciardo, S. Donatelli, and P. Kemper. Complexity of Kronecker operations on sparse matrices with applications to the solution of Markov models. ICASE Report 97-66 (NASA/CR-97-206274), Institute for Computer Applications in Science and Engineering, Hampton, VA, 1997. Submitted for publication.
6. G. Ciardo. Petri nets with marking-dependent arc multiplicity: properties and analysis. In R. Valette, editor, *Application and Theory of Petri Nets 1994, Lecture Notes in Computer Science 815 (Proc. 15th Int. Conf. on Applications and Theory of Petri Nets, Zaragoza, Spain)*, pages 179–198. Springer-Verlag, June 1994.
7. G. Ciardo and A. S. Miner. SMART: Simulation and Markovian Analyzer for Reliability and Timing. In *Proc. IEEE International Computer Performance and Dependability Symposium (IPDS'96)*, page 60, Urbana-Champaign, IL, USA, Sept. 1996. IEEE Comp. Soc. Press.
8. G. Ciardo and A. S. Miner. Storage alternatives for large structured state spaces. In R. Marie, B. Plateau, M. Calzarossa, and G. Rubino, editors, *Proc. 9th Int. Conf. on Modelling Techniques and Tools for Computer Performance Evaluation, LNCS 1245*, pages 44–57, St. Malo, France, June 1997. Springer-Verlag.
9. G. Ciardo and M. Tilgner. On the use of Kronecker operators for the solution of generalized stochastic Petri nets. ICASE Report 96-35, Institute for Computer Applications in Science and Engineering, Hampton, VA, May 1996.

10. S. Donatelli. Superposed generalized stochastic Petri nets: definition and efficient solution. In R. Valette, editor, *Application and Theory of Petri Nets 1994, Lecture Notes in Computer Science 815 (Proc. 15th Int. Conf. on Applications and Theory of Petri Nets)*, pages 258–277, Zaragoza, Spain, June 1994. Springer-Verlag.
11. T. Kam. *State Minimization of Finite State Machines using Implicit Techniques*. PhD thesis, University of California at Berkeley, 1995.
12. P. Kemper. Numerical analysis of superposed GSPNs. *IEEE Trans. Softw. Eng.*, 22(4):615–628, Sept. 1996.
13. P. Kemper. Reachability analysis based on structured representations. In J. Billington and W. Reisig, editors, *Application and Theory of Petri Nets 1996, Lecture Notes in Computer Science 1091 (Proc. 17th Int. Conf. on Applications and Theory of Petri Nets, Osaka, Japan)*, pages 269–288. Springer-Verlag, June 1996.
14. C. Y. Lee. Representation of switching circuits by binary-decision programs. *Bell Syst. Techn. J.*, 38(4):985–999, July 1959.
15. E. Pastor and J. Cortadella. Efficient encoding schemes for symbolic analysis of Petri nets. In *Proc. Design Automation and Test in Europe*, Feb. 1998.
16. E. Pastor and J. Cortadella. Structural methods applied to the symbolic analysis of Petri nets. In *Proc. IEEE/ACM International Workshop on Logic Synthesis*, June 1998.
17. E. Pastor, O. Roig, J. Cortadella, and R. Badia. Petri net analysis using boolean manipulation. In R. Valette, editor, *Application and Theory of Petri Nets 1994, Lecture Notes in Computer Science 815 (Proc. 15th Int. Conf. on Applications and Theory of Petri Nets, Zaragoza, Spain)*, pages 416–435. Springer-Verlag, June 1994.
18. O. Roig, J. Cortadella, and E. Pastor. Verification of asynchronous circuits by BDD-based model checking of Petri nets. In G. De Michelis and M. Diaz, editors, *Application and Theory of Petri Nets 1995, Lecture Notes in Computer Science 935 (Proc. 16th Int. Conf. on Applications and Theory of Petri Nets, Turin, Italy)*, pages 374–391. Springer-Verlag, June 1995.
19. A. Srinivasan, T. Kam, S. Malik, and R. K. Brayton. Algorithms for discrete function manipulation. In *International Conference on CAD*, pages 92–95. IEEE Computer Society, 1990.
20. D. Zampunièris. *The Sharing Tree Data Structure, Theory and Applications in Formal Verification*. PhD thesis, Department of Computer Science, University of Namur, Belgium, 1997.